

Write a program to implement **Linear Search** to find a given element in a list of integers.

The program should traverse the array sequentially and determine whether the element is present or not.

Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).
3. An integer **key** to be searched.

Output Format:

- If the element is found, print:
Element found at location : X
(Location starts from 1)
- If the element is not found, print:
Element not found.

Sample Input 1:

5

30 40 50 20 10

20

Sample Output 1:

Element found at location : 3

Sample Input 2:

5

30 40 50 20 10

55

Sample Output 2:

Element not found.

For example:

Input	Result
5 30 40 50 20 10 20	Element found at location : 3
5 30 40 50 20 10 55	Element not found.

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2  int LinearSearch(int a[],int n, int key);
3  int main()
4  {
5      int n, a[10], i, key, pos;
6      scanf("%d", &n);
7      for(i = 0; i<n;i++)
8          scanf("%d", &a[i]);
9      scanf("%d",&key);
10     pos = LinearSearch(a,n,key);
11     if(pos != -1)
12         printf("Element found at location : %d", pos);
13     else
14         printf("Element not found.");
15 }
16 int LinearSearch(int a[],int n, int key)
17 {
18     int i;
19     for(i = 0;i<n;i++)
20         if(a[i] == key)
21             return i;
22     return -1;

```

	Input	Expected	Got	
✓	5 30 40 50 20 10 20	Element found at location : 3	Element found at location : 3	✓
✓	5 30 40 50 20 10 55	Element not found.	Element not found.	✓

Passed all tests! ✓

Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (sorted array elements).
3. An integer **key** to be searched.

Output Format:

- If the element is found, print:
`Element found at location : X`
- If the element is not found, print:
`Element not found.`

Sample Input 1:

```
5
10 20 30 40 50
30
```

Sample Output 1:

Element found at location : 2

Sample Input 2:

```
5
10 20 30 40 50
55
```

Sample Output 2:

Element not found.

For example:

Input	Result
5 10 20 30 40 50 30	Element found at location : 2
5 10 20 30 40 50 55	Element not found.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int BinarySearch(int a[],int n,int key);
3 int main()
4 {
5     int n ,a[10],i,key,pos;
6     scanf("%d", &n);
7     for(i=0;i<n;i++)
8         scanf("%d",&a[i]);
9     scanf("%d",&key);
10    pos= BinarySearch(a,n,key);
11    if(pos != -1)
12        printf("Element found at location : %d",pos);
13    else
14        printf("Element not found.");
15    return 0;
16 }
17 int BinarySearch(int a[],int n,int key)
18 {
19     int first,last,mid;
20     first = 0;
21     last = n - 1;
22     while(first <= last)
```

	Input	Expected	Got	
✓	5 10 20 30 40 50 30	Element found at location : 2	Element found at location : 2	✓
✓	5 10 20 30 40 50 55	Element not found.	Element not found.	✓

Passed all tests! ✓

Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (sorted array elements).
3. An integer **key** to be searched.

Output Format:

- If the element is found, print:
Element found at location : X
- If the element is not found, print:
Element not found.

Sample Input 1:

5
10 20 30 40 50
30

Sample Output 1:

Element found at location : 2

Sample Input 2:

5
10 20 30 40 50
55

Sample Output 2:

Element not found.

For example:

Input	Result
5 10 20 30 40 50 30	Element found at location : 2
5 10 20 30 40 50 55	Element not found.

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int BinarySearch(int a[],int n,int key);
3 int main()
4 {
5     int n ,a[10],i,key,pos;
6     scanf("%d", &n);
7     for(i=0;i<n;i++)
8         scanf("%d",&a[i]);
9     scanf("%d",&key);
10    pos= BinarySearch(a,n,key);
11    if(pos != -1)
12        printf("Element found at location : %d",pos);
13    else
14        printf("Element not found.");
15    return 0;
16 }
17 int BinarySearch(int a[],int n,int key)
18 {
19     int first,last,mid;
20     first = 0;
21     last = n - 1;
22     while(first <= last)

```

	Input	Expected	Got	
✓	5 10 20 30 40 50 30	Element found at location : 2	Element found at location : 2	✓
✓	5 10 20 30 40 50 55	Element not found.	Element not found.	✓

Passed all tests! ✓

Finish review

Fundamentals of Data Structures Using C-2025 Batch

n Saturday, 23 May 2026, 11:09 AM

e Finished

n Saturday, 23 May 2026, 11:21 AM

n 11 mins 22 secs

e PAVITHRA B 2025-BME

Problem Statement:

Write a program to sort a list of integers in ascending order using the Insertion Sort algorithm.

Insertion Sort works by taking one element at a time and inserting it into its correct position in the already sorted part of the array.

Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).

Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

5

30 40 50 20 10

Sample Output:

10 20 30 40 50

For example:

Input	Result
5 30 40 50 20 10	10 20 30 40 50

Answer: (penalty regime: 0 %)

```

10     {
11         arr[j+1]=arr[j];
12         j=j-1;
13     }
14     arr[j+1]=key;
15 }
16 }
17 int main()
18 {
19     int n;
20     if(scanf("%d",&n) !=1)
21         return 0;
22     int arr[n];
23     for(int i=0;i<n;i++){
24         scanf("%d",&arr[i]);
25     }
26     insertionSort(arr,n);
27     for(int i=0;i<n;i++){
28         printf("%d %s", arr[i],(i ==n-1)?"":"" );
29     }
30     return 0;
31 }

```

	Input	Expected	Got	
✓	5 30 40 50 20 10	10 20 30 40 50	10 20 30 40 50	✓

Passed all tests! ✓

Finish review

Question 1
Correct
Marked out of 1.00
Flag question

Write a program to sort a list of integers in **ascending order** using the **Insertion Sort** algorithm.

Input Format:

- Output Format:

- Sample Input:

Sample Output:

10 20 30 40 50

For example:

Input	Result
5	10 20 30 40 50
30 40 50 20 10	

Answer: (penalty regime: 0 %)

```

10     {
11         arr[j+1]=arr[j];
12         j=j-1;
13     }
14     arr[j+1]=key;
15 }
16 }
17 int main()
18 {
19     int n;
20     if(scanf("%d",&n) !=1)
21         return 0;
22     int arr[n];
23     for(int i=0;i<n;i++){
24         scanf("%d",&arr[i]);
25     }
26     insertionSort(arr,n);
27     for(int i=0;i<n;i++){
28         printf("%d %s", arr[i],(i ==n-1)?"":"");
29     }
30     return 0;

```

	Input	Expected	Got	
✓	5 30 40 50 20 10	10 20 30 40 50	10 20 30 40 50	✓

Passed all tests! ✓

Finish review

Quiz navigation

n	Saturday, 23 May 2026, 11:21 AM
te	Finished
n	Saturday, 23 May 2026, 11:30 AM
n	8 mins 16 secs
ie	PAVITHRA B 2025-BME

Write a program to sort a list of integers in **ascending order** using the **Selection Sort** algorithm.

Selection Sort works by repeatedly selecting the **smallest element** from the unsorted portion of the array and placing it at the correct position in the sorted portion.

Input Format:

- 1. An integer **n** representing the number of elements.
- 2. **n** space-separated integers (array elements).

Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

5
30 40 50 20 10

Sample Output:

10 20 30 40 50

For example:

Input	Result
5 30 40 50 20 10	10 20 30 40 50

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main()
3 {
4     int n,i,j,min_idx,temp;
5     if(scanf("%d", &n) !=1)
6         return 0;
7     int arr[n];
8     for(i=0;i<n;i++){
9         scanf("%d",&arr[i]);
10    }
11    for(i=0;i<n-1;i++)
12    {
13        min_idx = i;
14        for(j=i+1;j<n;j++)
15        {
16            if(arr[j]<arr[min_idx]){
17                min_idx=j;
18            }
19        }
20        temp=arr[min_idx];
21        arr[min_idx]=arr[i];
22        arr[i]=temp;
```

	Input	Expected	Got	
✓	5 30 40 50 20 10	10 20 30 40 50	10 20 30 40 50	✓

Passed all tests! ✓

Started on	Saturday, 23 May 2026, 11:21 AM
State	Finished
Completed on	Saturday, 23 May 2026, 11:30 AM
Time taken	8 mins 16 secs
Name	PAVITHRA B 2025-BME

Question 1

Correct

Marked out of 1.00

Flag question

Write a program to sort a list of integers in **ascending order** using the **Selection Sort** algorithm.

Selection Sort works by repeatedly selecting the **smallest element** from the unsorted portion of the array and placing it at the correct position in the sorted portion.

Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).

Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

5
30 40 50 20 10

Sample Output:

10 20 30 40 50

For example:

Input	Result
5 30 40 50 20 10	10 20 30 40 50

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int main()
3 {
4     int n,i,j,min_idx,temp;
5     if(scanf("%d", &n) !=1)
6         return 0;
7     int arr[n];
8     for(i=0;i<n;i++){
9         scanf("%d",&arr[i]);
10    }
11    for(i=0;i<n-1;i++)
12    {
13        min_idx = i;
14        for(j=i+1;j<n;j++)
15        {
16            if(arr[j]<arr[min_idx]){
17                min_idx=j;
18            }
19        }
20        temp=arr[min_idx];
21        arr[min_idx]=arr[i];
22        arr[i]=temp;

```

	Input	Expected	Got	
✓	5 30 40 50 20 10	10 20 30 40 50	10 20 30 40 50	✓

Passed all tests! ✓

[Finish review](#)

Quiz navigation

1

[Finish review](#)

Fundamentals of Data Structures Using C-2025 Batch

n Saturday, 23 May 2026, 11:30 AM

e Finished

n Saturday, 23 May 2026, 11:38 AM

n 7 mins 32 secs

e PAVITHRA B 2025-BME

Write a program to sort a list of integers in **ascending order** using the **Bubble Sort** algorithm.

Bubble Sort works by repeatedly comparing adjacent elements and swapping them if they are in the wrong order. This process continues until the array is fully sorted.

Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).

Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

5
30 40 50 20 10

Sample Output:

10 20 30 40 50

For example:

Input	Result
5 30 40 50 20 10	10 20 30 40 50

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main()
3 {
4     int n,i,j,temp;
5     if(scanf("%d",&n) !=1)
6         return 0;
7     int arr[n];
8     for(i=0;i<n;i++){
9         scanf("%d",&arr[i]);
10    }
11    for(i=0;i<n-1;i++){
12        for(j=0;j<n-i-1;j++){
13            if(arr[j]>arr[j+1]){
14                temp=arr[j];
15                arr[j]=arr[j+1];
16                arr[j+1]=temp;
17            }
18        }
19    }
20    for(i=0;i<n;i++){
21        printf("%d %s",arr[i],(i==n-1)?"":" ");
22    }
```

	Input	Expected	Got	
✓	5 30 40 50 20 10	10 20 30 40 50	10 20 30 40 50	✓

Passed all tests! ✓

Started on	Saturday, 23 May 2026, 11:30 AM
State	Finished
Completed on	Saturday, 23 May 2026, 11:38 AM
Time taken	7 mins 32 secs
Name	PAVITHRA B 2025-BME

Question 1

Correct

Marked out of
1.00

Flag question

Write a program to sort a list of integers in **ascending order** using the **Bubble Sort** algorithm.

Bubble Sort works by repeatedly comparing adjacent elements and swapping them if they are in the wrong order. This process continues until the array is fully sorted.

Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).

Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

5
30 40 50 20 10

Sample Output:

10 20 30 40 50

For example:

Input	Result
5 30 40 50 20 10	10 20 30 40 50

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main()
3 {
4     int n,i,j,temp;
5     if(scanf("%d",&n) !=1)
6         return 0;
7     int arr[n];
8     for(i=0;i<n;i++){
9         scanf("%d",&arr[i]);
10    }
11    for(i=0;i<n-1;i++){
12        for(j=0;j<n-i-1;j++){
13            if(arr[j]>arr[j+1]){
14                temp=arr[j];
15                arr[j]=arr[j+1];
16                arr[j+1]=temp;
17            }
18        }
19    }
20    for(i=0;i<n;i++){
21        printf("%d %s",arr[i],(i==n-1)?"":" ");
22    }
```

	Input	Expected	Got	
✓	5 30 40 50 20 10	10 20 30 40 50	10 20 30 40 50	✓

Passed all tests! ✓

[Finish review](#)

Quiz navigation

1

[Finish review](#)

Fundamentals of Data Structures Using C-2025 Batch

n Saturday, 23 May 2026, 11:38 AM

e Finished

n Saturday, 23 May 2026, 11:49 AM

n 10 mins 50 secs

e PAVITHRA B 2025-BME

Write a program to sort a list of integers in **ascending order** using the **Quick Sort** algorithm.

Quick Sort is a **divide and conquer** technique that works by selecting a **pivot element**, partitioning the array into two subarrays, and recursively sorting them.

Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).

Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

5
30 40 50 20 10

Sample Output:

10 20 30 40 50

For example:

Input	Result
5 30 40 50 20 10	10 20 30 40 50

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 void swap(int *a,int *b)
3 {
4     int t=*a;
5     *a = *b;
6     *b = t;
7 }
8 int partition(int arr[],int low,int high)
9 {
10     int pivot=arr[high];
11     int i= (low-1);
12     for(int j=low;j<=high-1;j++)
13     {
14         if(arr[j]<pivot)
15         {
16             i++;
17             swap(&arr[i],&arr[j]);
18         }
19     }
20     swap(&arr[i+1],&arr[high]);
21     return(i+1);
22 }
```

	Input	Expected	Got	
✓	5 30 40 50 20 10	10 20 30 40 50	10 20 30 40 50	✓

Passed all tests! ✓

CS23232-Fundamentals of Data Structures Using C-2025 Batch

Started on Saturday, 23 May 2026, 11:38 AM

State Finished

Completed on Saturday, 23 May 2026, 11:49 AM

Time taken 10 mins 50 secs

Name PAVITHRA B 2025-BME

Question 1

Correct

Marked out of
1.00

Flag question

Write a program to sort a list of integers in **ascending order** using the **Quick Sort** algorithm.

Quick Sort is a **divide and conquer** technique that works by selecting a **pivot element**, partitioning the array into two subarrays, and recursively sorting them.

Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).

Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

5
30 40 50 20 10

Sample Output:

10 20 30 40 50

For example:

Input	Result
5 30 40 50 20 10	10 20 30 40 50

Answer: (penalty regime: 0%)

```
1 #include <stdio.h>
2 void swap(int *a,int *b)
3 {
4     int t=*a;
5     *a = *b;
6     *b = t;
7 }
8 int partition(int arr[],int low,int high)
9 {
10     int pivot=arr[high];
11     int i= (low-1);
12     for(int j=low;j<=high-1;j++)
13     {
14         if(arr[j]<pivot)
15         {
16             i++;
17             swap(&arr[i],&arr[j]);
18         }
19     }
20     swap(&arr[i+1],&arr[high]);
21     return(i+1);
22 }
```

	Input	Expected	Got	
✓	5 30 40 50 20 10	10 20 30 40 50	10 20 30 40 50	✓

Passed all tests! ✓

Finish review

Quiz navigation

1

Finish review

Fundamentals of Data Structures Using C-2025 Batch

n Saturday, 23 May 2026, 11:49 AM
e Finished
n Saturday, 23 May 2026, 12:15 PM
n 25 mins 53 secs
e PAVITHRA B 2025-BME

Write a program to sort a list of integers in **ascending order** using the **Merge Sort** algorithm.
Merge Sort is a **divide and conquer** algorithm that divides the array into smaller subarrays, sorts them recursively, and then merges the sorted subarrays to produce the final sorted array.

Input Format:

- 1. An integer **n** representing the number of elements.
- 2. **n** space-separated integers (array elements).

Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

5
30 40 50 20 10

Sample Output:

10 20 30 40 50

For example:

Input	Result
5 30 40 50 20 10	10 20 30 40 50

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 #include<stdlib.h>
3 void merge(int arr[],int l,int m,int r)
4 {
5     int i,j,k;
6     int n1=m-l+1;
7     int n2=r-m;
8     int L[n1],R[n2];
9     for(i=0;i<n1;i++) L[i]=arr[l+i];
10    for(j=0;j<n2;j++) R[j]=arr[m+1+j];
11    i=0;j=0;k=l;
12    while(i<n1 && j<n2)
13    {
14        if(L[i]<=R[j])arr[k++]=L[i++];
15        else arr[k++]=R[j++];
16    }
17    while(i<n1)arr[k++]=L[i++];
18    while(j<n2)arr[k++]=R[j++];
19 }
20 void mergeSort(int arr[],int l,int r)
21 {
22     if(l<r)
```

	Input	Expected	Got	
✓	5 30 40 50 20 10	10 20 30 40 50	10 20 30 40 50	✓

Passed all tests! ✓

Finish review

Started on	Saturday, 23 May 2026, 11:49 AM
State	Finished
Completed on	Saturday, 23 May 2026, 12:15 PM
Time taken	25 mins 53 secs
Name	PAVITHRA B 2025-BME

Question 1

Correct

Marked out of
1.00

Flag question

Write a program to sort a list of integers in **ascending order** using the **Merge Sort** algorithm.

Merge Sort is a **divide and conquer** algorithm that divides the array into smaller subarrays, sorts them recursively, and then merges the sorted subarrays to produce the final sorted array.

Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).

Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

5
30 40 50 20 10

Sample Output:

10 20 30 40 50

For example:

Input	Result
5 30 40 50 20 10	10 20 30 40 50

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 #include<stdlib.h>
3 void merge(int arr[],int l,int m,int r)
4 {
5     int i,j,k;
6     int n1=m-l+1;
7     int n2=r-m;
8     int L[n1],R[n2];
9     for(i=0;i<n1;i++) L[i]=arr[l+i];
10    for(j=0;j<n2;j++) R[j]=arr[m+1+j];
11    i=0;j=0;k=l;
12    while(i<n1 && j<n2)
13    {
14        if(L[i]<=R[j])arr[k++]=L[i++];
15        else arr[k++]=R[j++];
16    }
17    while(i<n1)arr[k++]=L[i++];
18    while(j<n2)arr[k++]=R[j++];
19 }
20 void mergeSort(int arr[],int l,int r)
21 {
22     if(l<r)
```

	Input	Expected	Got	
✓	5 30 40 50 20 10	10 20 30 40 50	10 20 30 40 50	✓

Passed all tests! ✓

[Finish review](#)

Quiz navigation

1

[Finish review](#)